root

Anyone Can Patch

Backporting CVEs with AI

root.io

Hi, Im John

CTO and co-Founder at Root

root

Why do we care about patching anyway?

The Problem with Detect → Upgrade → Fix

Why the Old Model Breaks

Breaking Changes

Every upgrade introduces incompatible API changes that ripple through dependent services and codebases.

Dependency Cascades

One forced upgrade triggers a cascade of downstream changes across the dependency graph.

Regression Risk

New code ships new bugs. Every upgrade trades a known CVE for unknown regression risk.

Supply Chain Attacks Are Industrializing

The threat landscape has fundamentally shifted

The threat model has changed

- Attackers target packages, not applications directly
- One compromised package → millions of downstream victims
- Supply chain attacks are now automated at scale

The upgrade model is broken

- You can’t upgrade your way to safety
- When the upgrade is the attack, upgrading makes it worse

How the Supply Chain Gets Weaponized

Shai-Hulud (npm self-replicating worm, Sept–Nov 2025) · TeamPCP (cascading CI/CD compromise, March 2026)

① Steal Maintainer Credentials → ② Inject Malicious Payload into Package → ③ Publish to npm / PyPI Registry → ④ Developer / CI/CD Installs → ⑤ Payload Executes

SHAI-HULUD · Self-Replicating npm Worm

- Steals npm tokens + GitHub PATs from developer's local environment
- Auto-publishes infected versions of every package the developer maintains
- Each victim's npm token infects 100+ more packages — no C2 needed
- 796 packages · 20M+ weekly downloads
  500+ GitHub orgs' credentials exposed

Both weaponize trusted registries

TEAMPCP · Cascading CI/CD Credential Theft

- Compromise Trivy (vulnerability scanner) running inside CI/CD — harvest publishing tokens
- Use tokens: backdoor Checkmarx KICS, LiteLLM (95M+ downloads/month), Telnyx SDK
- 47 npm packages infected in 60 seconds via automated worm across @emilgroup / @opengov
- Credential sweeper + .pth file executes on every Python startup · RSA C2 · Ransomware pivot

The Control Point: Pinned Versions

root

But Pinned Versions Still Have CVEs

Stability without security is just frozen vulnerability

Pinning gives you:

- Reproducible builds — no surprise version changes
- A clear control point for your environment

But CVEs still accumulate

- Pinned versions don’t self-heal
- The fix requires an upgrade you can’t take
- Without backporting, pinning becomes a liability

Backporting: The Missing Mechanism

What Is Backporting?

Apply a fix from a newer version to an older one

The situation:

- CVE fixed in v2.0. Your environment is locked to v1.8.

What backporting does:

- Extracts only the security fix from v2.0
- Applies it directly to your pinned v1.8
- Security benefit. Zero version bump.

Not a new idea:

- Debian, Red Hat, Ubuntu have shipped backported patches for decades
- Now applies to npm, pypi, and application packages

What Backporting Actually Is

Extract

Isolate the security logic from a fix in a later version. Ignore the noise: refactors, style changes, unrelated features.

Recontextualize

Port that logic into the structure of your pinned version. Adapt function names, paths, and code patterns as needed.

Preserve

Preserve existing behavior while removing the vulnerability. Minimal diff. No surprises.

Why Backporting Is Hard

Isolate the Fix

The fix commit contains refactoring, style changes, and unrelated features mixed with the security logic. Separating signal from noise is step one.

Bridge the Gap

Functions may be renamed, files moved, APIs redesigned. Adapting the fix to an older version requires understanding both codebases deeply.

Prove It Works

A patch that builds but does not close the vulnerability is worse than nothing. Validation requires understanding what the exploit actually does.

A New Model: Detect → Patch → Validate

Detect

Identify the CVE in your pinned dependency. Know exactly which version is vulnerable and what the attack vector is.

Patch

Backport the security fix directly into your pinned version. Minimal change. No upgrade required.

Validate

Build, test, and verify. Confirm the vulnerability is closed and no regressions were introduced.

root

Why This Matters

Stability

Your environment stays on known, tested versions. No surprise behavioral changes. No forced downstream migrations.

Minimal Change

Only the security-relevant code changes. The diff is small, auditable, and reviewable. No refactoring, no feature additions.

Control

You decide when to apply what fix. The pinned version remains the control point. Security and stability are no longer in conflict.

What You'll Do Today

1. Research

Find the CVE fix commit. Assess complexity. Determine if it is backportable to your target version.

2. Apply

Backport the security-relevant changes only. Adapt to the target version. Generate a unified diff.

3. Validate

Build the patched package. Run the test suite. Security-review the patch. Get a verdict.

Let’s Do This Live

root

DEMO TARGET

Demo: apache-airflow 2.10.5 + CVE-2025-68675

Apache airflow · pypi · Vulnerable: 2.10.5 · Fixed: 2.11.1

Complexity: LOW · 1 files · 2 lines

The vulnerability:

- **CVE-2025-68675** is an **information exposure** issue in Apache Airflow: the **proxy** and **proxies** fields on a **Connection** can hold proxy URLs that include **embedded credentials**

The goal:

- Research → patch → validate in under 30 minutes

https://nvd.nist.gov/vuln/detail/CVE-2025-68675

Three Prompts, One Patch

Prompt 1: Research

Validates CVE status, searches NVD + GitHub + upstream in parallel, analyzes fix commits, assesses backport complexity.

Research Output

Output: JSON with fix commit SHA, complexity score (1-10), and PROCEED / CAUTION / BLOCKED recommendation.

Prompt 2: Apply

Extracts security-only changes, adapts to target version, generates unified diff. No commit or push.

Prompt 3: Validate

Builds the package, runs the test suite, reviews the patch. Returns APPROVED / NEEDS_REVIEW / REJECTED.

Workshop Flow

Demo

Watch the three-prompt workflow live on axios + CVE-2025-27152. Understand how each prompt builds on the previous one.

Guided Practice

Work through a pre-selected CVE with the prompts in front of you. Use the JSON outputs to track what was found and changed.

Open Exploration

Pick a CVE from the extra credit list and run the full workflow. Green = straightforward. Work up to the harder ones.

https://github.com/Root-IO-Labs/anyone-can-patch

Extra Credit: Try These CVEs

1. CVE-2026-32871 — fastmcp 2.12.4 (Python)
2. CVE-2026-21441 — urllib3 2.6.0 (Python)
3. CVE-2024-47081 — requests 2.32.3 (Python)
4. CVE-2026-29086 — hono 4.11.7 (Node)
5. CVE-2026-27699 — basic-ftp 5.0.5 (Node)

References

1. root.io/blog/how-root-patches-actually-work
2. root.io/blog/agentic-vulnerability-remediation-fix-in-place-at-scale
3. root.io/blog/software-supply-chain-attacks-from-proof-of-concept-to-industrialized-threat
4. root.io/blog/when-scanners-fail-why-cve-feeds-and-vex-matter-more-than-ever
5. root.io/blog/the-hidden-cost-of-forced-upgrades
6. root.io/blog/cve-exposure-in-dependencies

root

You don’t need to upgrade everything to be secure.

Apply the right fix in the right place.